

WFLOW-03: Alert Notification Basics

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Sending Notifications with Workflows

The most common workflow use case is sending alert notifications to Slack, Microsoft Teams, and email. This notebook covers setting up connections, configuring notification tasks, and best practices for effective alerting.

Table of Contents

- [1. Notification Architecture](#)
 - [2. Setting Up Connections](#)
 - [3. Slack Notifications](#)
 - [4. Microsoft Teams Notifications](#)
 - [5. Email Notifications](#)
 - [6. Message Formatting](#)
 - [7. Complete Alert Workflow Example](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	automation:workflows:write , automation:connections:write
Prior Knowledge	WFLOW-01 and WFLOW-02
External Access	Slack workspace admin or Teams channel access

1. Notification Architecture

How Workflow Notifications Work

```

Trigger (Detected Problem, Schedule, etc.)
    ↓
Workflow Executes
    ↓
Notification Task
    ↓
Connection (credentials)
    ↓
External Service (Slack, Teams, Email)

```

Components

Component	Purpose
Connection	Securely stores credentials (tokens, webhooks)
Notification Task	Sends message using connection
Message Template	Dynamic content using Jinja expressions

Supported Notification Channels

Channel	Task Type	Authentication
Slack	<code>dynatrace.slack:slack-send-message</code> (Send message), <code>dynatrace.slack:request-approval</code> (Request approval)	OAuth App or Webhook
Microsoft Teams	<code>dynatrace.msteams:send-message</code>	Power Automate or Webhook (deprecated)
Email	<code>dynatrace.email:send-email</code>	Built-in — sends from <code>no-reply@apps.dynatrace.com</code> ; no SMTP setup
PagerDuty	<code>dynatrace.pagerduty:send-event</code> (Events API v2), <code>dynatrace.pagerduty:create-incident</code> (REST API)	Events connection (routing key) for Send event; REST API key for the others
ServiceNow	<code>dynatrace.servicenow:snow-create-incident</code> , plus other <code>snow-*</code> actions	OAuth or Basic Auth
Jira	<code>dynatrace.jira:jira-create-issue</code> , plus other <code>jira-*</code> actions	API Token
Custom Webhook (HTTP Request)	<code>dynatrace.automations:http-function</code>	Credential Vault (Basic or Token)

These action identifiers were checked on 10/02/2026 against Dynatrace's published workflow samples and connector templates. Once an action has run, the platform records its app and function in `dt.system.events` (`dt.automation_engine.action.app` / `.action.function`), so a tenant can confirm them too. The YAML in this series uses these identifiers and the actions' real input names (`message` , `content` , `payload` , `connectionId`). It is simplified in two places: connection names stand in for connection IDs, and named `conditions:` lists stand in for each task's `conditions` (`states` / `custom`). Export a workflow built in the editor to get the exact document shape.

Sources: [threat-detection-notification-sender.yaml \(Dynatrace GitHub\)](#) — "action: dynatrace.slack:slack-send-message", "action: dynatrace.msteams:send-message"; [wftpl_sample_servicenow_incident_man.yaml \(Dynatrace GitHub\)](#) — "action: dynatrace.servicenow:snow-create-incident"; [Workflow samples action catalog \(Dynatrace GitHub\)](#); [PagerDuty Connector actions \(DT docs\)](#) — "Trigger, acknowledge, or resolve an alert in PagerDuty using the Events API v2"; [Email \(DT docs\)](#).

2. Setting Up Connections

Connections store credentials separately from workflows for security and reusability.

Accessing Connections

1. Open **Settings** (gear icon)
2. Navigate to **Integration** → **Connections**
3. Or use direct URL: `https://<env>/ui/apps/dynatrace.hub/connections`

Creating a Connection

1. Click **+ Connection**
2. Select connection type (Slack, Teams, etc.)
3. Enter credentials
4. Name the connection (e.g., `slack-production-alerts`)
5. Save

Connection Best Practices

Practice	Description
Descriptive names	<code>slack-prod-oncall</code> not <code>slack1</code>
Environment separation	Different connections for prod/staging
Least privilege	Minimum required scopes
Regular rotation	Update tokens periodically

3. Slack Notifications

Option 1: Slack App (Recommended)

Provides richer features: channels, DMs, reactions, threads.

Setup:

1. Create Slack App at <https://api.slack.com/apps>
2. Add OAuth scopes: `chat:write` , `chat:write.public`
3. Install to workspace
4. Copy Bot User OAuth Token
5. Create connection in Dynatrace with token

Option 2: Incoming Webhook (Simpler)

Single channel, no advanced features.

Setup:

1. Go to Slack channel settings → Integrations
2. Add **Incoming Webhook**
3. Copy webhook URL
4. Create connection with webhook URL

Basic Slack Message Task

```
name: send_slack_alert
action: dynatrace.slack:slack-send-message
input:
  connection: slack-production      # the Slack connection's ID in an exported
workflow
  channel: "#alerts-production"
  messageFormat: slack_format
  message: |
    :rotating_light: *Problem Detected*

    *Title:* {{ event()["event.name"] }}
    *Category:* {{ event()["event.category"] }}
    *Started:* {{ event()["event.start"] }}

    &lt;{{ problem_link() }}|View in Dynatrace&gt;
```

Slack Message with Blocks

For richer formatting, put a Block Kit payload in `message`. The Send message action has no separate `blocks` input: with `messageFormat: slack_format`, *"Slack Markdown or Slack Block Kit inputs are processed"* from the message field.

```
input:
  connection: slack-production
  channel: "#alerts"
  messageFormat: slack_format
  message: |
    {
      "blocks": [
        { "type": "header",
          "text": { "type": "plain_text", "text": "{{ ':red_circle:' if
(event().get('event.severity') | int(5)) &lt;= 1 else ':large_orange_circle:'
}} {{ event()['event.category'] }} Alert" } },
        { "type": "section",
          "fields": [
            { "type": "mrkdown", "text": "**Problem:**\n{{ event()['event.name']
}}" },
            { "type": "mrkdown", "text": "**Status:**\n{{ event()['event.status']
}}" } ] },
        { "type": "section",
          "fields": [
            { "type": "mrkdown", "text": "**Started:**\n{{ event()['event.start']
}}" },
            { "type": "mrkdown", "text": "**ID:**\n{{ event()['display_id'] }}" }
] },
        { "type": "actions",
          "elements": [
            { "type": "button", "text": { "type": "plain_text", "text": "View
Problem" }, "url": "{{ problem_link() }}" } ] }
      ]
    }
```

The expressions resolve before the JSON reaches Slack, so a problem title that contains a double quote breaks the payload. Build the layout in Slack's Block Kit Builder and send one test message before you rely on it.

Sources: [Slack Connector actions \(DT docs\)](#) — *"If messageFormat is slack_format, Slack Markdown or Slack Block Kit inputs are processed."*; [Workflow samples action catalog \(Dynatrace GitHub\)](#) — *"Slack messages can be plain text/Slack Markdown, or a JSON Block Kit payload"*.

4. Microsoft Teams Notifications

Warning — O365 Connectors Deprecated: Microsoft retired Office 365 Connectors (Incoming Webhooks) in late 2024. Existing webhooks may continue to work temporarily, but **new O365 Connector creation is disabled**. Use one of these alternatives instead:

Method	Status	Notes
Power Automate Workflow	Recommended	Create a "When a Teams webhook request is received" flow in Power Automate; use the resulting URL in an HTTP Request task (<code>dynatrace.automations:http-function</code>)
Teams Workflows Connector	Recommended	Available in new Teams client under channel ... → Workflows
O365 Incoming Webhook	Deprecated	Legacy method shown below for reference only

Setting Up Teams Webhook (Legacy)

1. Open target Teams channel
2. Click ... → **Connectors** (or **Workflows** in new Teams)
3. Add **Incoming Webhook**
4. Name it (e.g., "Dynatrace Alerts")
5. Copy the webhook URL
6. Create connection in Dynatrace

Basic Teams Message Task

```
name: send_teams_alert
action: dynatrace.msteams:send-message
input:
  connectionId: teams-production    # the Teams connection's ID in an exported
  workflow
  message: |
    **Problem Detected**

    **Title:** {{ event()["event.name"] }}
    **Category:** {{ event()["event.category"] }}
    **Started:** {{ event()["event.start"] }}

    [View in Dynatrace]({{ problem_link() }})
```

Teams Adaptive Card

For richer formatting, paste Adaptive Card JSON into `message`. The Teams action has no separate card input.

```
input:
  connectionId: teams-production
  message: |
    {
      "type": "AdaptiveCard",
      "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
      "version": "1.4",
      "body": [
        { "type": "TextBlock", "text": "{{ event()['event.category'] }}"
Alert", "size": "Large", "weight": "Bolder",
        "color": "{{ 'Attention' if (event().get('event.severity') | int(5))
&lt;= 1 else 'Warning' }}" },
        { "type": "FactSet", "facts": [
          { "title": "Problem", "value": "{{ event()['event.name'] }}" },
          { "title": "Status", "value": "{{ event()['event.status'] }}" },
          { "title": "Started", "value": "{{ event()['event.start'] }}" } ]
        }
      ],
      "actions": [
        { "type": "Action.OpenUrl", "title": "View in Dynatrace", "url": "{{
problem_link() }}" }
      ]
    }
```

Use Dynatrace expressions for the dynamic values, as above. The docs say *"We don't support Adaptive Cards Template Language templating"*.

Sources: [Microsoft Teams Connector \(DT docs\)](#); [threat-detection-notification-sender.yaml \(Dynatrace GitHub\)](#) — *"action: dynatrace.msteams:send-message"*; [Workflow samples action catalog \(Dynatrace GitHub\)](#).

5. Email Notifications

Email Options

Option	Setup	Best For
Send email action (<code>dynatrace.email:send-email</code>)	No setup — sends from <code>no-reply@apps.dynatrace.com</code> ; the workflow needs the <code>email:emails:send</code> permission	Most notifications
Corporate mail relay	Not an option of the Send email action — call the relay's HTTP API from an HTTP Request or JavaScript task	A corporate sender address is required
SendGrid/SES	HTTP Request task to the provider's API	High volume, HTML templates

Basic Email Task

```
name: send_email_alert
action: dynatrace.email:send-email
input:
  to:
    - "oncall@company.com"
    - "platform-team@company.com"
  cc: []
  bcc: []
  subject: "[{{ event()['event.category'] }}] [{{ event()['event.name'] }}"
  content: |
    A problem has been detected in your Dynatrace environment.

    Problem Details:
    -----
    Title: [{{ event()["event.name"] }}]
    Category: [{{ event()["event.category"] }}]
    Status: [{{ event()["event.status"] }}]
    Start Time: [{{ event()["event.start"] }}]
    Problem ID: [{{ event()["display_id"] }}]

    Affected Entities:
    [{{ event()["affected_entity_ids"] | join(", ") }}]

    Root Cause:
    [{{ event()["root_cause_entity_id"] }}]

    View this problem:
    [{{ problem_link() }}]
```

Formatted Email

The `content` field takes markdown: bold, headings, lists, tables and links. It does not take HTML, so an HTML template arrives as literal tags, and there is no content-type input. For an HTML template, send through SendGrid or SES from an HTTP Request task (table above).

```

input:
  to: ["oncall@company.com"]
  cc: []
  bcc: []
  subject: "[{{ event()['event.category'] }}] [{{ event()['event.name'] }}"
  content: |
    ## [{{ event()['event.category'] }}] alert

    | | |
    |---|---|
    | **Problem** | [{{ event()['event.name'] }}] |
    | **Status** | [{{ event()['event.status'] }}] |
    | **Started** | [{{ event()['event.start'] }}] |

    [View in Dynatrace]([{{ problem_link() }}])

```

Sources: [Email \(DT docs\)](#) — "It doesn't offer support for HTML."; [threat-detection-notification-sender.yaml \(Dynatrace GitHub\)](#) — "action: dynatrace.email:send-email".

6. Message Formatting

Jinja Expression Reference

Expression	Result	Use
<code>[{{ event()["event.name"] }}]</code>	Problem title	Main content
<code>[{{ event()["event.category"] }}]</code>	AVAILABILITY/ERROR/SLOWDOWN/...	What kind of problem
<code>[{{ event().get("event.severity") \ int(5) }}</code>	1 (most severe) ... 5	Color coding
<code>[{{ event()["affected_entity_ids"] \ join(", ") }}</code>	Comma-separated list	Show all affected

Expression	Result	Use
<code>{{ event()["event.start"] }}</code>	ISO timestamp	When started
<code>{{ problem_link() }}</code>	Full URL	Link to problem (Problem trigger only)

`event.severity` is `experimental` in the semantic dictionary and is a 1–5 scale; the problem record carries no `CRITICAL / HIGH` strings. `int(5)` treats a missing severity as least severe — see WFLOW-04 §3 for why that default matters from SaaS 1.348.

Conditional Formatting

```
{% set sev = event().get("event.severity") | int(5) %}
{% if sev <= 1 %}
:red_circle: CRITICAL ALERT
{% elif sev == 2 %}
:large_orange_circle: HIGH ALERT
{% else %}
:large_yellow_circle: {{ event()["event.category"] }} ALERT
{% endif %}
```

Severity Emoji Map

<code>event.severity</code>	Slack Emoji	Teams Color
1 (Critical)	<code>:red_circle:</code>	Attention
2 (High)	<code>:large_orange_circle:</code>	Warning
3 (Medium)	<code>:large_yellow_circle:</code>	Accent
4 (Low)	<code>:large_blue_circle:</code>	Good

Inline Severity Mapping

```
{{ {1: ":red_circle:", 2: ":large_orange_circle:", 3: ":large_yellow_circle:",  
4: ":large_blue_circle:"}.get(event().get("event.severity") | int(5),  
":white_circle:") }}
```

7. Complete Alert Workflow Example

A production-ready workflow that sends alerts to Slack, Teams, and email.

Workflow Configuration

```
name: production-alert-notifications
description: Send alerts to all channels for production problems

trigger:
  type: davis-problem
  config:
    entityTagsMatch: all
    entityTags:
      - key: env
        value: prod

tasks:
  - name: slack_notification
    action: dynatrace.slack:slack-send-message
    input:
      connection: slack-production
      channel: "#alerts-production"
      message: |
        {{ {1: ":red_circle:", 2: ":large_orange_circle:", 3:
        ":large_yellow_circle:".get(event().get("event.severity") | int(5),
        ":white_circle:") }} *{{ event()["event.category"] }} Problem*

        *{{ event()["event.name"] }}*

        • *Status:* {{ event()["event.status"] }}
        • *Started:* {{ event()["event.start"] }}
        • *ID:* {{ event()["display_id"] }}

        &lt;{{ problem_link() }}|:mag: View Problem&gt;

  - name: teams_notification
    action: dynatrace.msteams:send-message
    input:
      connectionId: teams-production
      message: |
        **{{ event()["event.category"] }} Problem**

        **{{ event()["event.name"] }}**

        - **Status:** {{ event()["event.status"] }}
        - **Started:** {{ event()["event.start"] }}
        - **ID:** {{ event()["display_id"] }}

        [View Problem]({{ problem_link() }})
```

```
- name: email_notification
  action: dynatrace.email:send-email
  input:
    to: ["platform-oncall@company.com"]
    cc: []
    bcc: []
    subject: "[{{ event()['event.category'] }}] [{{ event()['event.name'] }}]"
    content: |
      A {{ event()["event.category"] }} problem has been detected.

      Problem: {{ event()["event.name"] }}
      Status: {{ event()["event.status"] }}
      Started: {{ event()["event.start"] }}

      View: {{ problem_link() }}
```

Task Execution

By default, tasks execute **in parallel**. All three notifications send simultaneously.

Monitor Your Notification Workflows

```
// Notification workflow executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
// and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
// any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
// `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
// `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
//   WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
//   send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
//   DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
//   is ERROR
//   .state.is_final                    true only on terminal records –
//   filter on it, otherwise a
//                                       single execution is counted once as
//   RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
//   task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
//   | limit 1
//   fetch dt.system.events, from:-24h
//   | filter event.kind == "WORKFLOW_EVENT"
//   | filter event.type == "ACTION_EXECUTION"
//   | filter dt.automation_engine.state.is_final == true
//   | filter in(dt.automation_engine.action.app, {"dynatrace.email",
//   "dynatrace.slack", "dynatrace.servicenow"})
//   | summarize executions = count(), by:{dt.automation_engine.action.app,
//   dt.automation_engine.state}
//   | sort executions desc
```

```

// Notification task success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state
== "SUCCESS")}, by:{dt.automation_engine.action.function}
| fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
| sort total desc
| limit 20

```

```
// Failed notification tasks
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.action.function,
dt.automation_engine.task.name, dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

Next Steps

Now that you can send basic notifications, learn advanced patterns:

Recommended Path

1. **WFLOW-04: Advanced Notification Routing** - Conditional routing by severity, team, time
2. **WFLOW-05: PagerDuty & ServiceNow** - Incident management integration
3. **WFLOW-06: Custom Notification Templates** - Rich formatting and dynamic content

Key Takeaways

- **Connections** store credentials securely and are reusable
 - **Slack** supports both OAuth apps and webhooks
 - **Teams** requires Power Automate or Teams Workflows connector (O365 Incoming Webhooks are deprecated)
 - **Email** can use built-in SMTP or custom servers
 - **Jinja expressions** enable dynamic message content
 - Tasks execute in **parallel** by default
-

Summary

In this notebook, you learned:

- Notification architecture: triggers → tasks → connections → external services
 - How to set up connections for Slack, Teams, and email
 - Basic and advanced Slack message formatting
 - Microsoft Teams notifications with Adaptive Cards
 - Plain-text and markdown email notifications (the Send email action does not render HTML)
 - Jinja expressions for dynamic content
 - A complete multi-channel alert workflow
-

References

- [Notification actions umbrella \(DT docs\)](#)
 - [Slack action \(DT docs\)](#)
 - [Microsoft Teams action \(DT docs\)](#)
 - [Email action \(DT docs\)](#)
 - [PagerDuty action \(DT docs\)](#)
 - [Workflow reference / Jinja expressions \(DT docs\)](#)
 - [Alerting and notifications umbrella \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.